CMF 3D-QSAR

The continuous molecular fields approach, as a program

Desktop application, library and command-line tool
for the Continuous Molecular Fields approach to 3D-QSAR
User manual

`github.com/igbaskin/cmf-qsar`

Contents

What this program is **5**

The idea in one page 5
What you can do with it 5
How this manual is arranged 6

How a prediction is made **7**

1. The fields 7
2. The overlap integral 7
3. Normalisation and scaling 8
4. Mixing the fields 8
5. Fitting: what the training set buys 8
6. Predicting a new molecule 9
The same prediction, as a field 9
A worked example 10
What changes for the other tasks 10
What has to match between fitting and predicting 11

Installation and first run **12**

Where the program lives 12
Requirements 12
Starting the application 12
A first model in five minutes 12

The application window **14**

The Data set panel 14
The Model panel 15
The 3D view 16
The Results panel 17
The menus 18

Preparing the data **19**

What the program reads 19
Activities 19
Alignment 19
Charges and per-atom properties 20

Building a model **21**

Choosing the fields 21
GRID-like probe fields 21
Descriptors that are not fields 22
Attenuation, regularisation and the grids 24
The learners 25
Nonlinear kernel layers and multiple kernel learning 25
Validation 25

Is it chance? y-randomisation 26
Where does the model apply? 26
How certain is that number? 27
Where the validation appears 27
Statistics reported 27
Saving and reusing 28

Reading the model 29

Co-fields 29
Contributions 29
Shapley values 29
A pharmacophore out of the co-field 30
What a candidate is missing 30
Chemical space maps 30

Classification and virtual screening 31

Setting up a classification task 31
One-class models: screening with actives alone 31
Decoys and the figures that matter 31

Conformations: the fourth dimension 33

Working without an alignment 34

Why the frame is the problem 34
The pose-free count kernel 34
The rotation-invariant autocorrelation kernel 34
Topomer fields 35
What the three routes are worth, measured 35
Heterogeneous series, step by step 36

Protein pockets 37

Getting a pocket on screen 37
Complementarity as numbers 37
Docking 37

Command-line reference 38

The options of `build`, by group 38
Worked examples 39

Using it as a library 40

The example data sets 41

When something goes wrong 42

Appendix A — Field parameterisation 43

Appendix B — Files and tags the program understands 44

Appendix C — Bibliography 45

The method 45

One-class classification and virtual screening 45
Methods and data the program borrows 45

What this program is

CMF 3D-QSAR is a desktop application, a Python library and a command-line tool implementing the **Continuous Molecular Fields** approach to three-dimensional structure–activity modelling. The method was introduced by Zhokhova, Baskin and co-workers (Dokl. Chem. **429** (2009) 273; J. Comput.-Aided Mol. Des. **27** (2013) 427; continuous indicator fields: J. Comput.-Aided Mol. Des. **29** (2015) 233; co-fields: Future Med. Chem. **11** (2019) 2701), and the program follows those papers closely enough that their published statistics can be reproduced from the same data — `docs/benchmark.md` records that comparison set by set.

The idea in one page

A molecule is represented not by a table of descriptors and not by values on a lattice, but by **continuous functions of space**. Each field is a sum of Gaussians centred on the atoms,

$$X_f(\mathbf{r}) = \sum_l w_{lf} \exp\left(-\frac{\alpha_f}{2} |\mathbf{r} - \mathbf{r}_l|^2\right)$$

where w_{lf} is the weight of atom l in field f — its partial charge for the electrostatic field, its Crippen logP contribution for the hydrophobic field, 1 for an indicator field of a given atom type, or any number the file carries per atom — and α_f is the attenuation factor that sets how far the field reaches.

Two molecules are compared by the **overlap integral** of their fields, which for Gaussians has a closed form:

$$K_f(A,B) = \int X_{fA}(\mathbf{r})\, X_{fB}(\mathbf{r})\, d\mathbf{r} = \left(\frac{\pi}{\alpha}\right)^{3/2} \sum_l \sum_m w_{lf} w_{mf} \exp\left(-\frac{\alpha}{4} |\mathbf{r}_l - \mathbf{r}_m|^2\right)$$

No grid is ever evaluated for this: the kernel is a sum over atom pairs, computed exactly. The kernels of the individual fields are mixed with coefficients h_f into one kernel, and that kernel goes into a kernel method — ridge regression, a support vector machine, a Gaussian process. Three consequences follow, and they are what distinguishes the approach from CoMFA:

- **Nothing is discretised.** There is no lattice spacing to choose, no alignment of grids, no variable selection among tens of thousands of columns, and no truncation at a cut-off distance.
- **The model is a function of space again.** A kernel ridge model predicts $y(X) = \Sigma_j a_j K(X, X_j) + b$, and because the kernel is an integral of a product, the whole sum can be rewritten as the overlap of the molecule's field with one function, the **co-field** $C_f(\mathbf{r}) = \Sigma_j a_j X_{fj}(\mathbf{r})$. The co-field is the program's map: it plays the role of CoMFA's coefficient contour, but it is the model rather than a projection of it.
- **The decomposition is exact.** Because the prediction is bilinear in the fields, it can be split without approximation into contributions of training molecules, of field types, of regions of space, or of individual atoms.

What you can do with it

Task	How
Build a 3D-QSAR regression model on an aligned congeneric series	Model panel, or `cmf_cli.py build`
Work without an alignment	topomer fields, the rotation-invariant autocorrelation kernel, or the pose-free count kernel
Classify actives against inactives, or screen with actives alone	binary (SVC, ν-SVC, Laplace GP) and one-class (1-SVM, centroid, k-NN, GP, kernel PCA) tasks
See what the model says about space	co-fields as isosurfaces, molecular fields, overlap maps, Gaussian-cube export
See what the model says about a molecule	training-molecule contributions, atom contributions, Shapley values and interaction indices
Turn a model into design instructions	pharmacophore extraction from co-field lobes, field-difference maps against a reference ligand
Use the protein	fetch or load a pocket, draw it behind the fields, score field complementarity, dock
Average over conformations	conformational ensembles (4D) and multi-instance learning
Map a data set	kernel GTM chemical-space maps with activity, class and density landscapes

How this manual is arranged

Chapter 2 follows one prediction from the atoms to the number, which is the method in full. Chapters 3 and 4 get you running and name every part of the window. Chapters 5 to 7 are the working cycle: prepare the data, fit a model, read it. Chapters 8 to 11 are the specialised routes — screening, conformations, alignment-free modelling, protein pockets. Chapter 12 is the complete command-line reference, chapter 13 the library, chapter 14 the example data, and chapter 15 collects the things that go wrong. The appendices hold the field parameterisation, the file conventions and the bibliography.

Conventions: *Menu → Item* names a menu command, `monospace` names a file, an option or a piece of code. Equations are quoted in the notation of the papers, with their equation numbers.

How a prediction is made

This chapter follows one number — the activity predicted for one molecule — from the atoms it starts in to the value that appears in the table. Nothing here is a sketch: every formula is the one the program evaluates, and the worked example at the end was computed by running it.

The chain has six links:

1. each molecule becomes a set of **fields**, one per field type, each a sum of Gaussians on its atoms;
2. every pair of molecules gets a **field kernel** per field type, the overlap integral of those two fields, computed in closed form;
3. each field kernel is **normalised** and **scaled**;
4. the field kernels are **mixed** into one kernel matrix;
5. the learner turns that matrix and the activities into **dual coefficients**;
6. a new molecule is pushed through steps 1 to 4 against the training set, and its prediction is the **weighted sum of its kernel values** with those coefficients.

1. The fields

Atom l of a molecule carries a weight w_{lf} in field f — its partial charge in the electrostatic field, its Crippen logP contribution in the hydrophobic field, an Abraham-type increment in the hydrogen-bond fields, the Tripos van der Waals well depth in the steric field, 1 in an indicator field for atoms of that type and 0 elsewhere, or a number read from the file in a `prop:` field. Appendix A lists them.

The field itself is the sum of Gaussians those weights define,

$$X_f(\mathbf{r}) = \sum_l w_{lf} \exp\left(-\frac{b_{lf}}{2}|\mathbf{r}-\mathbf{r}_l|^2\right)$$

with width $b_{lf} = \alpha_f$ for every field except the steric one, where it is scaled by the atom's van der Waals radius, $b_{lf} = \alpha_f / R_l^2$, so that a large atom spreads its bulk over a correspondingly larger region.

There is no grid. The field is never evaluated anywhere in order to make a prediction; it is evaluated only when a map has to be *drawn*.

2. The overlap integral

Two Gaussians have an overlap integral in closed form, so the inner product of two whole fields is a double sum over atom pairs:

$$K_f(A,B) = \int X_{fA}(\mathbf{r}) X_{fB}(\mathbf{r})\, d\mathbf{r} = \sum_{l\in A}\sum_{m\in B} w_{lf} w_{mf} \left(\frac{2\pi}{b_{lf}+b_{mf}}\right)^{3/2} \exp\left(-\frac{b_{lf} b_{mf}}{2(b_{lf}+b_{mf})}|\mathbf{r}_l-\mathbf{r}_m|^2\right)$$

When every atom shares one width, $b_{lf} = b_{mf} = \alpha$, this collapses to the form the papers quote,

$$K_f(A,B) = \left(\frac{\pi}{\alpha}\right)^{3/2} \sum_l \sum_m w_{lf} w_{mf} \exp\left(-\frac{\alpha}{4}|\mathbf{r}_l-\mathbf{r}_m|^2\right)$$

and the general form is what the steric field needs, since its widths differ atom by atom. A unit test checks both against numerical quadrature of the fields on a fine grid.

Two things follow immediately. The cost of a kernel is the number of atom pairs, not the number of grid points, which is why a 300-molecule set takes seconds. And the attenuation α is the only thing that decides how far apart two atoms may be and still contribute: large α demands close matching, small α lets distant atoms see each other, and in the limit α → 0 the geometry drops out altogether and the kernel becomes a product of atom counts — which is the alignment-free baseline of *Working without an alignment*.

3. Normalisation and scaling

Two corrections stand between the raw integrals and a usable kernel, and they do different jobs.

Cosine normalisation (*Normalise kernels*) divides each field kernel by the norms of the two fields,

$$\hat{K}_f(A,B) = \frac{K_f(A,B)}{\sqrt{K_f(A,A)\,K_f(B,B)}}$$

so that every molecule has unit self-similarity in every field and a large molecule cannot look important merely for being large. $\hat{K}_f(A,A) = 1$ by construction.

Field scaling (*Scale fields*) then multiplies each field kernel by a constant c_f chosen so that its mean diagonal over the training set is 1. It equalises the fields' contributions before they are mixed, which matters when one field's raw magnitudes dwarf another's — an electrostatic kernel built from partial charges and a steric kernel built from well depths are not naturally on the same scale. With cosine normalisation on, every diagonal is already 1 and $c_f = 1$; the two options are therefore alternatives in practice, and unticking both gives the raw-kernel convention of the papers.

The scaling constants are computed **on the training set and then stored**: a prediction must not rescale anything according to the molecules being predicted, or two test molecules would be scored on different scales.

4. Mixing the fields

The joint kernel is a weighted sum over field types,

$$K(A,B) = \sum_f h_f\, c_f\, \hat{K}_f(A,B)$$

with the mixing coefficients h_f left at 1 by default, edited by hand in the field table, chosen by coordinate descent on the cross-validated error, or set by centred kernel alignment. Since each normalised field kernel has unit diagonal, the joint self-kernel of any molecule is $\Sigma_f\, h_f\, c_f$ — five, for the five default fields with unit coefficients, which is the 5 on the diagonal of the matrix in the worked example below.

5. Fitting: what the training set buys

Kernel ridge regression with a bias term solves one linear system (Eq. 12 of the 2013 paper):

$$(\mathbf{K} + \lambda \mathbf{I})\,\mathbf{a} + b\,\mathbf{1} = \mathbf{y}, \qquad \mathbf{1}^T\mathbf{a} = 0$$

The outcome is one **dual coefficient** a_j per training molecule and one bias b. The last row of the system is the constraint $\Sigma_j\, a_j = 0$, which is what makes the bias meaningful: the coefficients describe only *departures* from a constant level, and b carries the level itself. The ridge parameter λ adds to the diagonal, which keeps the system solvable when molecules are nearly identical — as they are in a congeneric series, where the off-diagonal kernel values run above 0.97 — and controls how hard the model chases the training activities.

The other learners differ only in how **a** and b are obtained, never in what they are: ε-SVR and ν-SVR get them from a quadratic program in which most a_j come out exactly zero, C-SVC and ν-SVC the same for class labels, and the Laplace Gaussian-process classifier from the posterior mode of a latent function. In every case the fitted model is the pair (**a**, b) over the training molecules.

6. Predicting a new molecule

For a molecule the model has never seen, the program:

1. builds its fields **with the same field set** — the same α, the same charge model, the same parameterisation, the same atom typing;
2. computes the per-field overlap integrals between its atoms and the atoms of every training molecule, giving one row $K_f(x, j)$ per field;
3. normalises that row with the molecule's own self-kernels and the **stored** training self-kernels, and multiplies by the **stored** scales c_f;
4. mixes the fields into one row $k(x)_j = \Sigma_f\, h_f\, c_f\, K_f(x, j)$;
5. returns

$$\hat{y}(x) = \sum_j a_j\, k(x)_j + b = \mathbf{k}(x)^T \mathbf{a} + b$$

That is the whole prediction: a similarity-weighted combination of the training activities, where "similarity" is the overlap of molecular fields and the weights were fixed when the model was fitted. Molecules with $a_j = 0$ — most of them, after an SVM fit — contribute nothing, which is why the *Contributions* tab can list the handful that actually make a given prediction.

The same prediction, as a field

Because the kernel is an integral of a product and the prediction is linear in it, the sum over training molecules can be moved inside the integral. Writing $s_j = 1/\sqrt{\{K_f(j,j)\}}$ for the normalisation of training molecule j, define the **co-field**

$$C_f(\mathbf{r}) = \sum_j a_j\, s_j X_{fj}(\mathbf{r})$$

and the prediction becomes

$$\hat{y}(x) = \sum_f \frac{h_f c_f}{\sqrt{K_f(x,x)}} \int X_{fx}(\mathbf{r})\, C_f(\mathbf{r})\, d\mathbf{r} + b$$

The model is therefore a *function of space*, one per field type, and a molecule is scored by how much of its own field lies in the positive regions of that function and how much in the negative ones. This is the identity that makes the pictures of *Reading the model* the model rather than an

illustration of it, and the division by the molecule's own field norm is the only thing the normalisation adds to the picture. It also explains what co-fields cannot survive: a nonlinear kernel layer, which is not an integral of a product, and the autocorrelation kernel source, which has no frame to draw in.

A worked example

Four molecules from `examples/benzamides_synthetic.sdf` as the training set, the five default fields, α = 0.5, cosine normalisation, λ = 0.1, and the fifth molecule as the one to predict. The numbers below are the program's own, printed from the objects it fits.

The joint kernel of the training set, $K = \Sigma_f K_f$ over five fields, so the diagonal is 5:

```
              parent     4F      4'F     4Cl          y
parent        5.0000  2.8362  3.2195  2.5368       5.585
4F            2.8362  5.0000  2.3310  3.9647       5.230
4'F           3.2195  2.3310  5.0000  2.8882       6.471
4Cl           2.5368  3.9647  2.8882  5.0000       5.355
```

Solving the bordered system gives

```
a = [-0.1358, -0.0581, +0.3487, -0.1549]      sum(a) = 0      b = 5.7124
```

The test molecule 4'Cl is then compared with each training molecule, field by field:

```
                      parent      4F      4'F     4Cl
electrostatic         0.0878  0.6028  0.1009  0.3915
steric                0.9793  0.9824  0.9863  0.9579
hydrophobic           0.8936  0.9412  0.9443  0.8437
H-bond acceptor       0.2600  0.9895  0.1927  0.6539
H-bond donor          0.5815  1.0000  0.4424  0.7379
                      ------  ------  ------  ------
k(x)  (their sum)     2.8022  4.5159  2.6667  3.5849
```

and the prediction is that row times the coefficients, plus the bias:

```
y(x) = 2.8022(-0.1358) + 4.5159(-0.0581) + 2.6667(0.3487) + 3.5849(-0.1549) + 5.7124
     = 5.4444
```

which is exactly what *Pred (test)* shows for that molecule. Two things are worth reading off this example rather than taking on trust. The steric and hydrophobic rows are nearly constant across the training set — these molecules differ by one halogen — so almost all the discrimination comes from the electrostatic and hydrogen-bond fields, which is visible in the spread of their rows. And the prediction, 5.44 against an observed 7.21, is poor, because four training molecules and no hyper-parameter search is not a model: the arithmetic is the lesson here, not the accuracy.

What changes for the other tasks

Model	What the prediction is
Kernel ridge	$\hat{y} = \mathbf{k}^T\mathbf{a} + b$, as above
Gaussian process	the same mean, plus the variance $\sigma^2(x) = K(x,x) - \mathbf{k}^T(\mathbf{K}+\lambda\,\mathbf{I})^{-1}\mathbf{k}$, which the *Align by prediction confidence* tool minimises

Model	What the prediction is
ε-SVR, ν-SVR	the same formula; most a_j are zero
C-SVC, ν-SVC	the same formula, read as a decision value: its sign is the class, its magnitude the margin
Laplace GP classifier	the same formula gives the latent mean, which is squashed to a probability and averaged over the latent variance
One-class methods	a score built from the same kernel row, but also from the molecule's own self-kernel: distance to the centroid, to the k-th nearest active, to the principal subspace, or the 1-SVM decision value
Nonlinear kernel layer	the joint kernel is passed through $\exp(-\gamma\, d^2)$ or an arc-cosine function first; the prediction formula is unchanged, the co-field identity is lost
Autocorrelation source	the kernel row comes from the rotation-invariant distance profiles instead of the overlap integrals; everything downstream is identical

What has to match between fitting and predicting

Because the kernel row is computed against the stored training atoms with the stored constants, a prediction is only meaningful if the new molecule was prepared the way the training molecules were. The saved `.cmf` model therefore carries the field set, the α values, the charge model, the parameterisation, the normalisation and scaling flags, the mixing coefficients, the training atoms and the coefficients — everything in steps 1 to 5 — and reuses them. Two things it cannot enforce, and which are yours to get right:

- **The frame.** The overlap integral is not invariant to rotation or translation, so a molecule predicted in a different frame from the training set will score low on every field and be predicted towards the mean. If the new molecules are not already in that frame, align them (*Preparing the data*) or use a frame-free kernel source (*Working without an alignment*).
- **The chemistry behind the weights.** Charges read from a file for the training set and Gasteiger charges for the new molecules are two different electrostatic fields wearing one name.

Installation and first run

Where the program lives

The repository is github.com/igbaskin/cmf-qsar — private, so the link asks for a GitHub account with access:

```
git clone https://github.com/igbaskin/cmf-qsar.git
cd cmf-qsar
```

Everything this manual describes is in that checkout: the `cmfqsar` package, the application (`cmf_app.py`), the command-line tool (`cmf_cli.py`), the example data under `examples/`, the benchmark scripts under `benchmarks/` with their recorded results, the tests, and this manual with the script that typesets it.

Requirements

Python 3.10 or newer, and the packages in `requirements.txt`:

```
pip install -r requirements.txt
```

That brings RDKit (chemistry), PySide6 (the interface), pyqtgraph with PyOpenGL (the 3D view), scikit-image (marching cubes for the isosurfaces), scikit-learn (the SVM solvers), NumPy, SciPy and joblib. The program is pure Python and needs no compilation step; it has been used on Windows, Linux and macOS.

Optional extras, only needed for particular features:

- `dimorphite_dl` — protonation states when importing SMILES for a given pH;
- `matplotlib` — supplies the DejaVu fonts used to typeset this manual;
- a local ORCA or VASP installation if you intend to compute per-atom quantum properties to use as fields (see *Preparing the data* and appendix A).

Starting the application

```
python cmf_app.py
```

or straight onto a data set, which saves two dialogs:

```
python cmf_app.py examples/benzamides_synthetic.sdf --activity pActivity --test-fraction 0.25
```

If the 3D view stays black, the OpenGL driver is the usual cause; see *When something goes wrong*.

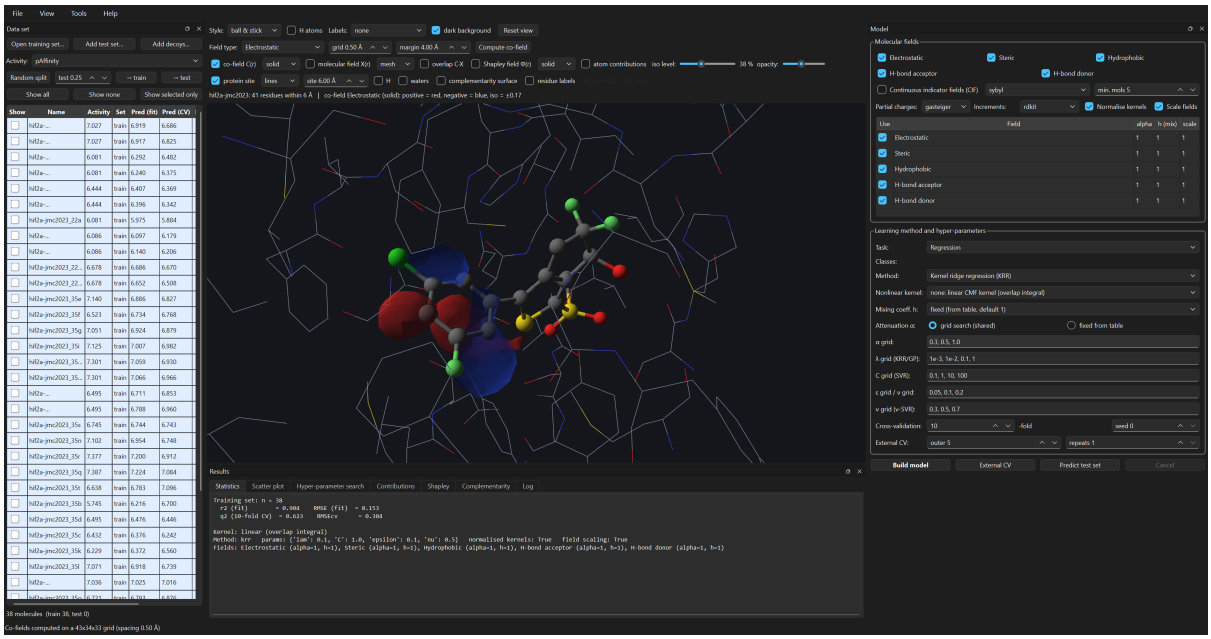
A first model in five minutes

1 *File → Open training set…* and choose `examples/benzamides_synthetic.sdf`. It holds 109 aligned N-phenylbenzamides with a synthetic activity, so you know the answer the model should find. Pick `pActivity` as the activity property.

2. In the data-set table, press *Random split* and set a quarter of the molecules aside as a test set.
3. In the Model panel leave the five physico-chemical fields ticked, leave the method at kernel ridge regression, and press **Build model**.
4. Read the *Statistics* tab: q^2 should be about 0.87 and R^2_{pred} about 0.84 — the noise ceiling of that set is 0.94, so the model has recovered nearly all of the signal that was put into it.
5. Press **Compute co-field**, choose *Electrostatic*, and move the *iso level* slider. The red and blue lobes that appear are the pseudo-receptor zones the activity was generated from.

Nothing in that sequence is specific to the example: it is the working cycle of the program.

The application window



The HIF-2α series of the Merck free-energy benchmark, 38 ligands: a kernel ridge model with $q^2 = 0.62$, its electrostatic co-field drawn as ± isosurfaces, and the binding site of the reference crystal structure, which the data set brings with it and which needs no superposition.

The window is a 3D view with three dockable panels around it. The **3D view** and its control strip occupy the centre and are always there; the **Data set** panel docks on the left, the **Model** panel on the right inside a scroll area, and the **Results** panel across the bottom under the view. All three are ordinary dock widgets: drag one by its title bar to another edge, drop it on another to stack them as tabs, or pull it off entirely to float it on a second monitor. The layout is remembered between sessions, and *View → Restore default layout* puts it back when an experiment goes wrong.

At the bottom, the **status bar** shows what the program is doing, with a progress bar on the right during a long task. Every long task — a grid search, a co-field, an alignment, a conformer generation — runs in a worker thread, so the window stays responsive and the *Cancel* button in the model panel stops the work rather than the application.

The Data set panel

Three buttons at the top load structures: **Open training set…**, **Add test set…** and **Add decoys…**, the same actions as the first three entries of the *File* menu. Beside them the *Activity* selector names the SD property the activities are read from; changing it re-reads the column without reloading the file.

The table has one row per molecule and seven columns:

Column	Contents
Show	the tick that decides whether this molecule is drawn in the 3D view
Name	the molecule's title from the file
Activity	the experimental value, from the chosen property

Column	Contents
Set	train, test or decoy
Pred (fit)	the model's prediction for the molecule it was fitted on
Pred (CV)	its prediction when it was in the held-out fold
Pred (test)	its prediction as a test molecule
AD	its similarity to the training set, when the applicability domain was computed

An eighth column, **Chemotype**, appears once the chemotypes have been found. Rows are selected singly or in blocks, and the selection drives everything else: the co-field overlap, the atom contributions, the Shapley decomposition and the complementarity numbers all refer to the molecule currently selected.

Sorting. Clicking a column header sorts by it, a second click reverses the order, and a third restores the order of the file. Molecules with no value in that column always sink to the bottom, whichever direction is in force, so an unpredicted molecule never hides among the predictions. The *Show* ticks, the selection and the 3D view follow the molecule and not the row, so sorting never disturbs what is on screen.

Splitting. *Random split* assigns a fraction to the test set; the → *train* and → *test* buttons move the selected rows by hand. A `subset`, `set` or `split` tag in the file is honoured on loading, which is how the published splits of the example sets arrive without any clicking.

Showing. *Show all*, *Show none* and *Show selected only* control the view in bulk. With one molecule shown, the maps in the viewer refer to it; with all of them shown, the co-field sits in the middle of the series as it should.

A line under the table reports the state of the data set — how many molecules, how many in each subset, which activity property — and a note in brackets records any transformation that has been applied, for instance *[topomer substituents of …]*, so that a changed data set never passes for the original.

The Model panel

Everything that defines a model, in the order one normally works down it. The panel scrolls; on a small screen nothing is hidden, it is merely below.

Molecular fields. Five tick boxes for the physico-chemical fields — Electrostatic, Steric, Hydrophobic, H-bond acceptor, H-bond donor — then *Continuous indicator fields (CIF)* with its typing scheme (`sybyl`, `element`, `element+aromatic`) and the minimum number of training molecules an atom type must occur in to earn a field. Below them, *Per-atom property fields* lists the per-atom properties the loaded file carries, each of which can be turned into a field of its own.

Partial charges chooses between Gasteiger, MMFF94 and the charges stored in the file; *Increments* chooses the parameterisation (`rdkit`, `tripos`, `tripos-legacy`). Two tick boxes set the kernel conventions: *Normalise kernels* for cosine-normalised field kernels, *Scale fields* to rescale every field kernel to unit mean diagonal before mixing — untick it for the raw-kernel convention of the papers.

Conformations switches between one conformation per molecule and an ensemble, with the weighting (uniform or Boltzmann), kT, a cap on the number of conformations, and the multi-instance settings (the softness β, and *one conformation* for the hard selection). A label beside

them reports how many conformations the data set currently holds.

A table at the foot of the group lists the fields that will actually be built, with the columns **Use**, **Field**, **alpha**, **h (mix)** and **scale**: per-field attenuation and mixing coefficients are editable there, which is how a field is given more or less weight than the others without leaving the panel.

Learning method and hyper-parameters. A form, row by row:

Row	What it sets
Task	regression, binary classification, or one-class (virtual screening)
Classes	for classification: a threshold on the activity, with *active below* for data where small is potent, or a class property from the file
Method	the learner; the list changes with the task — KRR, GP, ε-SVR, ν-SVR for regression; C-SVC, ν-SVC, Laplace GP for classification; 1-SVM, centroid, k-NN, one-class GP and kernel PCA for one-class work
Nonlinear kernel	none (the linear CMF kernel), a Gaussian layer, or one or two arc-cosine layers
Mixing coeff. h	fixed from the table, optimised by coordinate descent, or set by centred kernel alignment
Attenuation α	a single value, or the grid below
Kernel from	overlap integrals of the fields, which need one frame, or the rotation-invariant autocorrelation, which does not
α grid, λ grid, C grid, ε/ν grids	the search ranges, as comma-separated lists
Cross-validation	the number of folds and the shuffling seed
External CV	the outer folds and the number of repeats for the nested procedure
Probe fields	GRID-like fields for the water, methyl, carbonyl, amine and cation probes, and the number of Gaussians per atom
Computed descriptors	molecular weight, clogP, TPSA and rotatable bonds, computed by RDKit; all off by default
Descriptors (per molecule)	which numeric molecule properties enter as non-field descriptors, and the block's mixing coefficient
Validation	*shuffles* for y-randomisation, *bootstrap* for confidence intervals, and the *applicability domain* tick — all three off by default, all three run as part of **Build model** rather than from a button of their own

Four buttons close the panel: **Build model** runs the grid search, the final fit, the internal cross-validation and the test prediction; **External CV** runs the nested procedure and reports q^2ex; **Predict test set** applies the current model to the test molecules; **Cancel** stops whatever is running.

The 3D view

The view itself is an OpenGL scene: drag to orbit, the wheel to zoom, middle-drag (or shift-drag) to pan. *Reset view* reframes the whole scene, *Tools → Center view on current molecule* reframes the selected one, and *File → Save 3D view as image…* writes what you see to PNG.

The control strip above the view is four rows, and everything on it redraws immediately:

- **Molecules.** Drawing style (ball & stick, sticks, lines, space fill), *H atoms*, *Labels* (none, element + number, atom type, partial charge), *dark background*, *Reset view*.

- **Maps.** *Field type* chooses which field the maps refer to; **Compute co-field** evaluates the co-fields on a grid around the training set; then the maps themselves — *co-field C(r)*, *molecular field X(r)*, *overlap C·X*, *Shapley field Φ(r)*, *atom contributions*, *difference map*, *pharmacophore* — each with its own solid-or-mesh style where that applies.
- **Levels.** *iso level*, the isosurface threshold as a percentage of the largest absolute value rather than an absolute number, so the surfaces stay comparable when the field changes; and *opacity*.
- **Protein site.** *protein site* draws the residues within the chosen distance of the molecules on screen, as lines or sticks, with *H*, *waters* and residue labels optional, and *complementarity surface* paints the molecule's surface by how well it complements the pocket.

Co-field colours are fixed, so that maps made in different sessions can be compared at a glance:

Field	Positive lobe	Negative lobe
Electrostatic	red	blue
Steric	green	yellow
Hydrophobic	orange	light blue
H-bond acceptor	magenta	dark red
H-bond donor	cyan	purple
Indicator field	red: the atom type is welcome here	blue: it is not

The Results panel

Twelve tabs, each filled by the work that produces it:

Tab	What it holds
Statistics	every figure of merit, for the fit, the cross-validation, the test set and the external CV; the y-randomisation, bootstrap and applicability-domain results follow at the bottom, under a *Validation* heading, when they were asked for
Scatter plot	observed against predicted, by subset, with the molecule under the cursor named
ROC	the receiver-operating curves of a classification model, per subset
Scores	the ranked decision values or calibrated probabilities: rank, molecule, score, set, class
Enrichment	the enrichment curve against a decoy set, with EF at 0.5, 1, 5 and 10 %, BEDROC and RIE
Map	the kernel-GTM chemical-space map, with the activity, class and density landscapes, and a control for how the molecules themselves are drawn over it
Hyper-parameter search	the whole grid — α, the other parameters, RMSEcv and q^2 — so that an optimum at the edge of the grid is visible rather than hidden
Contributions	two tables for the selected molecule: the training molecules that make its prediction, with `a_j` and `a_j K(X, X_j)`, and its own atoms with their contributions
Shapley	the Shapley decomposition, over atoms, substituents of the common scaffold or field types, with second-order interactions

Tab	What it holds
Complementarity	the field complementarity of the molecules and of the co-field with the pocket, by sign agreement, cosine or correlation
Chemotypes	the Murcko scaffolds found in the set, their sizes, and the leave-one-chemotype-out result beside the alignment-free baseline
Log	everything the program did, with timings and warnings — including the names of the molecules that fell outside the applicability domain — the first place to look when a result surprises you

The menus

File — *Open training set…* (Ctrl+O), *Add test set…*, *Add decoys (background set)…*, *Import activities from CSV…*; *Load protein structure (PDB/mmCIF)…*, *Fetch protein from the PDB…*, *Forget protein structure*; *Load model…* (Ctrl+L), *Save model…* (Ctrl+S); *Export predictions (CSV)…*, *Export aligned data set (SDF)…*, *Export current co-field (Gaussian cube)…*, *Export current Shapley field (Gaussian cube)…*, *Save 3D view as image…*; *Quit* (Ctrl+Q).

View — show or hide each panel, *Results panel spans full width* for when the tables and plots matter more than the view, and *Restore default layout*.

Tools — *Generate conformations (4D)…*; *Dock into the pocket…*; *Align data set to reference…*; *Align by prediction confidence (GP variance)…* and its cross-validated form for the training set; *Replace molecules by their topomer substituents…*; *Field difference from a reference molecule…*; *Extract pharmacophore from the co-field…*; *Chemical space map (GTM)…*; *Align decoys to the training set…*; *Unalign (randomise poses)…*; *Center view on current molecule*; and the submenu *Heterogeneous sets (several chemotypes)* with its four numbered steps.

Help — *About CMF 3D-QSAR / method summary*, which names the equations of the papers that each part of the program implements.

Every *Tools* entry opens a dialog that states what it is about to do and on which molecules, and each one runs in the background with the progress bar and the *Cancel* button live.

Preparing the data

What the program reads

Format	Notes
`.sdf`, `.sd`, `.mol`	3D coordinates used as they are; hydrogens are added with coordinates if missing
`.mol2`	Sybyl atom types and partial charges are read and can be used directly; a tolerant fallback reader handles blocks RDKit refuses
`.smi`, `.csv`, `.tsv`, `.txt`	2D input: each molecule is embedded with ETKDG and minimised with MMFF94, which means **the poses are arbitrary** until you align them
`.pdb`, `.mmcif`	protein structures only, read by a deliberately permissive reader

A CSV needs a column of SMILES; a column called `name` or the first non-SMILES column names the molecules, and every other column is kept as a molecule property, so activities and metadata travel with the file.

Activities

Three ways, in order of convenience:

1. **An SD property.** The loader guesses the activity property and the dialog lets you change it. Anything numeric will do: `pIC50`, `pKi`, `pActivity`, `CBG_pK`.
2. **A separate table** (*File → Import activities from CSV…*, `--activity-file`): molecule name, activity, and optionally a third column with the split (1 = train, 2 = test, or the words).
3. **A class property** for classification tasks, or a threshold on the activity.

A per-molecule tag named `subset`, `set` or `split` is honoured automatically, which is how the published splits of the example sets arrive.

Alignment

CMF compares fields in one frame of reference. If your molecules came from a superposition, from a crystallographic series or from a docking run, that frame already exists and you should leave it alone. If they came from SMILES, they have no frame at all and you must make one — or use one of the frame-free routes of *Working without an alignment*, which is often the better answer.

The tools, in increasing order of sophistication:

- *Tools → Align data set to reference…* — rigid superposition on the maximum common substructure with a reference molecule, optionally refined by maximising the CMF kernel similarity, which is the alignment criterion proposed in the 2013 paper.
- *Tools → Align by prediction confidence…* — minimises the Gaussian-process posterior variance $\sigma^2(x) = K(x,x) - k(x)^T(K + \lambda I)^{-1}k(x)$ of a molecule under an existing model, which moves it into the model's applicability domain. It uses no activity value, so it is valid for prospective prediction. The program refuses to move training molecules, because the optimum there is degenerate: superimposing everything maximises mutual similarity and destroys the signal.

- *Tools → Align decoys to the training set…* — the same idea for a background set, so that no decoy is rejected merely for having been posed badly.
- *Tools → Unalign (randomise poses)…* — the control experiment. It gives every molecule a random rigid pose, which leaves internal geometry and self-kernels untouched and scrambles only the mutual arrangement. Randomise, realign, and see how much comes back; the program remembers the poses from before and reports the recovery.

Charges and per-atom properties

The electrostatic field needs charges: Gasteiger (the default), MMFF94, or **read from the file**, which is what you want with the Sutherland sets and their MNDO ESP-fit charges, or with anything you computed yourself.

Beyond that, any per-atom number can become a field of its own. `prop:<name>` reads the atomic weights from a property of the molecule — a quantum charge, a Fukui index, a local softness, an atomic polarisability, even a per-atom embedding — and that field then behaves exactly like a built-in one, with its own attenuation, its own mixing coefficient, its own co-field, atom contributions and Shapley values.

Two conventions are read, in this order:

1. a property set on **every atom**, which RDKit writes to SDF as `atom.dprop.<name>` and reads back automatically;
2. a **molecule property holding one number per atom**, separated by spaces, commas or newlines, which is how most quantum-chemistry packages dump such a table.

The hydrogens are the trap. A file without 3D coordinates is embedded on loading and hydrogens are added, so a list written for the heavy atoms alone no longer matches the atom count. The program says so rather than silently mis-assigning: `--prop-missing error` (the default) stops, `--prop-missing zero` gives the unlisted atoms zero weight. `examples/orca_atom_properties.py` shows the whole route from an ORCA run to an SDF the program reads.

Building a model

Choosing the fields

The five physico-chemical fields — electrostatic, steric, hydrophobic, H-bond acceptor, H-bond donor — are the default and reproduce the published CMF models. The steric field is special: its width is scaled by the van der Waals radius of each atom (α/R^2_{vdW}, Eq. 10–11 of the 2013 paper) rather than being shared.

Continuous indicator fields (CIF, 2015 paper) add one field per atom type, with weight 1 on atoms of that type and 0 elsewhere. Tick *Continuous indicator fields* and set the minimum number of training molecules an atom type must appear in (3 by default) to be given a field of its own. Three typing schemes are available: `sybyl` (the types of the original scripts, read from MOL2 or assigned), `element`, and `element+aromatic`. CIFs are the most descriptive field set the program has, and on several of the benchmark sets they match or beat the physico-chemical fields on their own.

`prop:<name>` fields are added by naming them in the per-atom property row of the model panel, or in `--fields`.

GRID-like probe fields

Goodford's GRID describes a molecule by the energy a small probe — a water, a methyl, a carbonyl oxygen — would feel at every point around it. That is a field in exactly this program's sense, and it says something the built-in fields do not: not "there is hydrophobic character here" but "a probe placed here would be bound". *Probe fields* in the model panel (`--probes water,carbonyl`) adds one such field per probe. Five are defined:

name	what it stands for	R (Å)	well (kcal/mol)	charge	hydrogen bonding
`water`	water, OH_2	1.70	0.15	0	both, weakly
`methyl`	a methyl group	2.00	0.10	0	neither
`carbonyl`	a carbonyl oxygen	1.60	0.20	−0.40	accepts
`amine`	an amine nitrogen	1.75	0.16	+0.40	donates
`cation`	a generic +1 cation	1.50	0.10	+1.00	neither

Those four numbers are the whole definition of a probe: the radius and well depth enter the dispersion term through the usual combining rules, the charge the electrostatic term, and the hydrogen-bond character decides which atoms it is complementary to — a donor probe feels acceptor atoms and not other donors. The parameters sit in the usual GRID and Tripos ranges rather than being taken from a published GRID table, which matters if you ever want to compare numbers with GRID itself rather than shapes. Adding a probe is one entry in `PROBES` in probes.py; the fitting, the kernel, the drawing and both interfaces read that dictionary, so a new one appears everywhere at once.

The obstacle is arithmetic, not conceptual: a Lennard-Jones term is not a Gaussian, has no closed-form overlap, and diverges at the atom, so the integral of its square does not exist. The way through is to fit each atom's contribution by a few Gaussians, which keeps every kernel in the program a closed form over atom pairs. Three things had to be measured before that worked, and each one shaped what is implemented:

- **The repulsive wall cannot be carried.** Fitting a curve that is +30 kcal/mol at 2.5 Å and −0.13 at 3.5 Å with a handful of Gaussians produces coefficients of $\pm 10^{7}$ that cancel, and a worst-case error larger than the well it is meant to represent. The wall is a truncation artefact in every grid implementation anyway, and this program already carries it — the steric field *is* a Gaussian van der Waals envelope — so a probe field carries the **attraction** only.
- **A well at a radius is a shell**, and a shell is not a sum of Gaussians centred on the atom. Each term is therefore capped at its own optimum, which makes it monotone in r, and those a sum of Gaussians represents naturally.
- **The r^{-6} tail is not Gaussian either**, and is negligible beyond 6 Å, so the energy is tapered to zero between 5 and 7 Å exactly as GRID's own cut-off does.

What comes out reproduces the attraction to about 15 % in the volume-weighted norm the kernel sees (23 % on a realistic worst case), and the kernel it implies agrees with numerical integration of the fields themselves to $4 \cdot 10^{-7}$ — a test asserts both. Each atom carries K Gaussians (four by default), so a probe field costs K^2 passes over the distance matrix where an ordinary field costs one; three is the cheap setting and five the careful one.

Measured against the fields it would join, same molecules, same α and λ grids, 10-fold CV:

set	n	the five fields	+ water probe	probes only
GPB	66	+0.539	+0.539	+0.406
THER	76	+0.538	+0.543	+0.428
THR	88	+0.672	**+0.699**	+0.665
ACE	114	+0.647	+0.648	+0.607

So: adding a water probe to the five fields is worth between nothing and 0.027 q^2, and probe fields on their own are worse than the fields they would replace. That is the honest result and it was the expected one — CoMSIA replaced GRID-like energies with Gaussian similarity indices for much the same reason, and this program's fields are closer to CoMSIA than to GRID by design. What a probe field is actually *for* is interpretation: its co-field is drawn, exported and decomposed like any other, and it says where a water or a carbonyl oxygen would be bound, which is a more directly actionable statement than "more acceptor character here".

Two limitations are worth stating plainly. The directional part of GRID's hydrogen-bond term — the angles at the donor and at the acceptor's lone pairs — is **not** represented; the radial part is, broadened to 0.6 Å because a sharper shoulder is not representable by the shared basis. And alignment by field similarity refuses to run on a probe field rather than silently ignoring it, because the gradient of the expansion is not implemented.

Descriptors that are not fields

Everything above describes a molecule as a function of space. Some things are not functions of space and never will be: a measured logP, a molecular weight, the medium or temperature a series was recorded in, a count of rotatable bonds, an energy from a quantum calculation. A model that

cannot see them has to pretend the activity depends on geometry alone, which for an adsorption potency or a permeability is simply false.

The *Descriptors (per molecule)* row of the model panel (`--descriptors A,B`) offers every numeric molecule property the file carries — and hides itself when the file carries none besides the activity, which is why it is invisible on most of the example sets; the computed descriptors of the next section are always there. Ticking some adds one more block to the kernel:

$$K(A,B) = \sum_f h_f c_f \hat{K}_f(A,B) + h_d c_d \, \mathbf{z}(A) \cdot \mathbf{z}(B)$$

where **z** holds the descriptors of a molecule, standardised with the **training set's** mean and standard deviation so that a quantity in kilojoules cannot outvote one in log units, and h_d is a mixing coefficient like a field's (the *h* box beside the row, `--descriptor-h`; zero switches the block off without removing it).

Three properties of the block are deliberate:

- **It is scaled, not cosine-normalised.** Dividing by the two norms, as the fields are divided, would send a molecule sitting at the training mean in every descriptor — an unremarkable molecule — to a zero row and a zero diagonal, out of the model altogether. Instead the whole block is multiplied by a constant that puts its mean diagonal at 1, so ten descriptors do not outweigh the geometry merely by being ten.
- **The statistics belong to the training set** and are stored in the saved model. Standardising a test molecule by its own set would score two test sets on two different scales, which is the mistake the field scaling avoids as well.
- **The model stays linear in its inputs**, so the decomposition rules survive. A prediction splits into a field part and a descriptor part, and the descriptor part splits further into one number per descriptor, which is what the program reports.

A molecule missing a descriptor is refused by default (`prop_missing`), because a model that silently substitutes the mean is telling you nothing about that molecule while looking as confident as ever; with the `mean` policy it is substituted and the molecule simply contributes nothing through that descriptor. A descriptor that is constant over the training set is reported as inert rather than dividing by a spread of zero.

The obvious trap, stated plainly. A descriptor derived from the activity — the activity itself, a transform of it, a value fitted to it — will dominate the model and make the geometry look worthless, while q² goes up. The program cannot detect this, so it reports the mean absolute contribution of every descriptor beside the model statistics:

```
Non-field descriptors (mean |contribution| per training molecule):
  clogp                          0.0779
  mw                             0.0176
  tpsa                           0.0123
```

If one descriptor carries most of the prediction, the model is a descriptor model wearing a 3D coat, and the honest response is to report it as one.

```
python cmf_cli.py build series.sdf --activity pIC50 --descriptors clogp,mw,tpsa
```

Descriptors the program computes itself

A file that carries no numeric tags has nothing to offer the row above, so four descriptors are always available, computed by RDKit from the structure: `calc:mw` (molecular weight), `calc:clogp` (Wildman–Crippen logP), `calc:tpsa` (topological polar surface area) and `calc:rotatable` (rotatable bonds). They have a row of their own in the model panel, *Computed descriptors*, and **all four are off by default**.

Those four are shortcuts, not the whole list. **Any** of RDKit's descriptors — some two hundred of them — can be asked for by its own RDKit name, typed into the box at the end of that row (with completion) or passed on the command line: `calc:NumHAcceptors`, `calc:FractionCSP3`, `calc:BalabanJ`, `calc:LabuteASA`, `calc:qed`. Names are matched case-insensitively, and a name that does not exist is refused before the fit with the nearest ones that do — `calc:NumHDonor` answers *did you mean NumHDonors?*.

The default is deliberate, and so is the warning that belongs with it. These are two-dimensional quantities: they can lift a model that has a size or lipophilicity trend in it, and they can also carry a model that then owes nothing at all to its geometry. Measured on four Sutherland sets, same α and λ grids, 10-fold CV:

set	n	five fields	+ the four computed	the four alone
THR	88	**+0.671**	+0.634	+0.620
GPB	66	**+0.534**	+0.525	+0.494
ACE	114	+0.657	+0.707	**+0.712**
THER	76	**+0.509**	+0.496	+0.480

Read ACE carefully: four numbers that know nothing of the geometry predict better than the whole 3D model. That is not an argument for using them — it is a warning about that data set, and it agrees with what the alignment-free baseline already said about ACE elsewhere in this manual. On the other three sets they cost a little.

So use them to *test* a model rather than to improve it: if adding them changes nothing, the geometry is carrying the model; if they take it over, the 3D part was decoration. The contribution table printed beside the statistics is what tells you which happened.

Attenuation, regularisation and the grids

Two hyper-parameters matter. α sets how far each field reaches: large α makes the fields sharp, so molecules must match closely, while small α makes them diffuse. λ (or *C*, ε, ν for the SVM learners) sets the regularisation. Both are chosen by cross-validated grid search, and the whole grid is shown in the *Hyper-parameter search* tab — look at it, because an optimum at the edge of the grid means the grid was too small.

```
python cmf_cli.py build data.sdf --alphas 0.1,0.3,0.5,1,2,5 --lams 1e-3,1e-2,0.1,1,10
```

Refinements, all optional: continuous Brent minimisation over λ as in the original R scripts (`--lam-search brent`), a Nelder–Mead search over α (`--alpha-search nelder-mead`), coordinate descent or centred kernel alignment for the mixing coefficients `h_f` (`--optimize-h`, `--mkl`), and a grid over the bandwidth of a nonlinear kernel layer (`--gammas`).

Two conventions decide how the field kernels are combined, and they change the numbers: *Normalise kernels* uses cosine-normalised field kernels, and *Scale fields* rescales each field kernel

to unit mean diagonal before mixing. Untick the second to get the raw-kernel convention of the papers (`--raw-scale`).

The learners

Task	Methods	Key
Regression	kernel ridge regression with bias; Gaussian process (the same fit plus predictive variances); ε-SVR; ν-SVR	`krr`, `gp`, `svr`, `nusvr`
Binary classification	C-SVC; ν-SVC; Gaussian-process classifier by the Laplace approximation	`svc`, `nusvc`, `gpc`
One-class / screening	1-SVM; distance to the centre of the actives; distance to the k-th nearest active; one-class Gaussian process; distance from the principal subspace (kernel PCA)	`ocsvm`, `ocmean`, `ocknn`, `ocgp`, `ockpca`

Kernel ridge regression is the method of the papers and the default. The Gaussian-process view adds a posterior variance per prediction at no extra cost, which is what the confidence-based alignment of *Preparing the data* minimises. The Laplace GP classifier adds calibrated probabilities, a posterior variance and model selection by marginal likelihood — it scores no better than C-SVC on the benchmark sets (mean difference 0.001 in Matthews correlation), so it is not the default, but its probabilities are honest and its evidence picks hyper-parameters without holding anything out.

Nonlinear kernel layers and multiple kernel learning

A layer on top of the CMF kernel makes the model more expressive than an overlap integral: a Gaussian layer, $\exp(-\gamma\, d^2)$ with $d^2 = K(i,i) + K(j,j) - 2K(i,j)$, or one or two arc-cosine layers, the kernel of an infinitely wide neural network (Cho & Saul 2009). The price is interpretation: the model is then no longer linear in the field, so co-fields and atom contributions are unavailable. The similarity decomposition stays valid, and the **Shapley tab** decomposes the prediction instead, which is why it exists.

Multiple kernel learning chooses the mixing coefficients h_f rather than leaving them at 1, either by coordinate descent on the cross-validated error or by centred kernel alignment (`--mkl align|alignf`), which is a closed-form, training-only criterion.

Validation

- **k-fold cross-validation** (10 by default) gives q^2 and RMSEcv. Folds are shuffled unless you ask for the interleaved folds of the original scripts (`--cv-no-shuffle`).
- **A test set** gives R^2pred and RMSEpred. Both variance conventions are reported, because they differ for sets whose test activities span a narrow range — the steroid benchmark is the classic example, where the ten test affinities cover 1.9 log units and R^2pred computed against the test variance is negative while the predictions are good.
- **External (nested) cross-validation** (`--nested`, *External CV*) wraps the whole model selection in an outer loop and reports q^2ex and RMSEcvex. This is the number to quote: the ordinary q^2 has seen the hyper-parameter choice.

- **y-randomisation**, a **bootstrap** interval and the **applicability domain** are the three sections below.
- **Leave-one-chemotype-out** folds, for a set of several scaffolds, never leave a molecule's own scaffold-mates in the training part. Random folds flatter such sets badly — by 0.11 to 0.47 q^2 on the five chemotype benchmarks here — and the program reports both, together with an alignment-free fragment baseline that the aligned model has to beat.

Is it chance? y-randomisation

A cross-validated q^2 says how well the model predicts; it does not say whether a model of the same shape could have reached that number from activities that mean nothing. In a congeneric series, where kernel values above 0.97 are normal, that is a real possibility, and it is the first thing a referee asks about a 3D-QSAR model.

The **shuffles** box of the model panel's *Validation* row (`--y-scramble N`) shuffles the activities N times, refits and cross-validates each time, and reports where the real q^2 falls among the scrambled ones — the mean and spread of the shuffled values, the largest of them, how many standard deviations above them the real one lies, and the permutation p-value.

```
python cmf_cli.py build data.sdf --alphas 0.5 --lams 0.1 --y-scramble 100
```

Two properties of the test are worth knowing, and the program states both rather than letting the number mislead:

- **A permutation test cannot report a p below 1/(N+1).** Ten shuffles bottom out at p = 0.091, which is above every conventional threshold *however good the model is*. The program says so — "no shuffle came close, but 10 of them cannot support a p below 0.091" — instead of reporting a chance-correlation warning that the data do not support. Use 19 shuffles for $p < 0.05$ and 99 for $p < 0.01$.
- **The hyper-parameters are held fixed** by default, so the test asks whether the *fit* can be reached by chance. Tick **redo the search** beside the shuffles box (`--y-scramble-select`) for the stricter question — whether the whole procedure, model selection included, can be. It matters: on the steroid set with a 3 x 3 grid, the best scrambled q^2 rises from −0.08 to **+0.32** once the search is allowed to pick the luckiest grid point on noise, while the real value stays at +0.78. The kernels do not depend on the activities, so they are computed once and the extra cost is one cross-validation per grid point per shuffle.

Where does the model apply?

A kernel method always returns a number. For a molecule unlike anything in the training set, that number is the training mean in disguise, and nothing in q^2 or R^2pred reveals it.

The **applicability domain** tick of that row (`--domain`) measures how close each molecule outside the training set is to it, **in the kernel the model itself uses**: the cosine similarity of their fields, averaged over the *k* nearest training molecules (3 by default). The threshold is not a number chosen out of the air — it is taken from the training set's own spread, the 5th percentile of the same quantity computed among the training molecules, so a series of close analogues gets a tight domain and a diverse set a loose one. Molecules below it are listed by name in the log, their similarity appears in the data-set table's **AD** column — which sorts like any other, so the extrapolations can be brought to the top — and the exported predictions carry both the value and a yes/no verdict, so that an extrapolation stays marked after it leaves the program.

On the steroid benchmark, nine of the ten test compounds fall inside and one — 4-pregnene-3,11,20- trione — falls outside. On BZR, training on the largest chemotype and scoring the others, every molecule of the training chemotype is inside and 29 of 40 molecules from other chemotypes are outside, which is the behaviour one wants from a domain estimate.

The field-space distance is reported alongside the similarity, because the two can disagree: a molecule that is a scaled-up version of a training molecule points the same way (high cosine) but sits far away (large distance).

How certain is that number?

A difference of 0.02 in q^2 is quoted all over the benchmarks in this repository. Some of those differences are real and some are the particular molecules they were computed over.

The **bootstrap** box of that row (`--bootstrap N`) resamples the (observed, predicted) pairs N times and reports a percentile confidence interval for q^2 and for R^2pred. It is the uncertainty of the *statistic* given these predictions, which is exactly what decides whether two settings differ. On the 21 training steroids, $q^2 = 0.768$ carries a 95 % interval of [0.47, 0.89] — wide enough to settle any argument about a hundredth.

`--bootstrap-refit N` adds Cramer's bootstrap as well: resample the training set with replacement, refit, and score the molecules that were left out of the bag. That one measures the variability of the fit rather than of the statistic, and it is the more pessimistic of the two. The kernel is never recomputed — a resample is an index into the matrix — so a hundred refits cost about as much as one cross-validation.

Where the validation appears

The three checks of *Building a model* — y-randomisation, the bootstrap intervals and the applicability domain — are set in one place and reported in another, which is worth stating plainly because there is no button and no separate window for them.

Setting them: the *Validation* row at the foot of the model panel's second group, directly under *External CV*. Three controls: **shuffles** (0 = off), **bootstrap** (0 = off), and the **applicability domain** tick. They are settings, not actions — choose them before pressing **Build model**, and they run as part of that build, after the fit and the test prediction.

Reading them: the *Statistics* tab of the results panel, where they are appended below the usual figures under a heading *Validation*. The names of the molecules that fell outside the applicability domain go to the *Log* tab instead, with their similarity values, so that the statistics stay a summary and the list of offenders sits with the rest of the commentary.

Three cases produce no validation block at all, and each says why rather than leaving a blank:

- all three controls at their defaults — nothing was asked for;
- a classification or one-class model — y-randomisation and the bootstrap are implemented for regression only, and the block says so;
- the autocorrelation kernel source — the applicability domain is measured in the field kernel, which that source does not provide.

Statistics reported

Regression: r^2 and RMSE for the fit, q^2 and RMSEcv for the cross-validation, R^2pred and RMSEpred for the test set in both variance conventions, q^2ex and RMSEcvex for the external CV. Classification: accuracy, balanced accuracy, sensitivity, specificity, Matthews correlation, AUC and the ROC curve, each for the fit, the cross-validation and the test set. Screening: AUC, enrichment factors at 0.5, 1, 5 and 10 %, the best achievable enrichment at each of those fractions, BEDROC and RIE.

Saving and reusing

File → *Save model...* writes a `.cmf` file (joblib) holding the field set, the kernels' definition, the fitted coefficients, the training molecules and every setting needed to predict later. `cmf_cli.py predict model.cmf new.sdf --out predictions.csv` applies it; the GUI loads it with *File* → *Load model...*.

Reading the model

A CMF model is readable in a way a grid model is not, because the prediction can be taken apart exactly. The 2019 paper states three decomposition rules, and the program implements all three.

Co-fields

The co-field is the model as a function of space:

$$C_f(\mathbf{r}) = \sum_j a_j X_{fj}(\mathbf{r})$$

a weighted sum of the training molecules' fields with the fitted dual coefficients as weights. The prediction of any molecule is then the overlap of its own field with the co-field, field type by field type. Press **Compute co-field** and the program evaluates it on a grid around the training set; choose the field with the *Field* selector and set the *iso level* slider, which is a percentage of the largest absolute value rather than an absolute number, so the surfaces stay meaningful when you change field.

What the lobes mean: a positive co-field lobe is a region where *more* of that field raises the predicted activity. For the electrostatic field that means positive charge is favoured there, for the steric field bulk, for the hydrophobic field lipophilicity, and for an indicator field it means an atom of that type is welcome at that position.

Two further maps use the same grid, for the molecule currently shown:

- the **molecular field** $X_f(\mathbf{r})$ of that molecule, in the same colours, drawn as a mesh by default so that it can be seen together with the co-field;
- the **overlap** $C_f(\mathbf{r})\, X_f(\mathbf{r})$, whose integral *is* that field's contribution to the prediction (Eq. 7), so the map shows where the prediction comes from.

File → Export current co-field (Gaussian cube) writes a cube file readable by PyMOL, VMD or Jmol, which is the way to put a co-field into a publication figure.

Contributions

The *Contributions* tab answers two questions about the selected molecule:

- **Which training molecules** make its prediction, as $a_j\, K(X, X_j)$ — the similarity decomposition, rule 2. A prediction resting on one or two training molecules is a warning.
- **Which of its atoms**, as the atom contributions of Eq. 14 of the 2015 paper, drawn in the 3D view as red and blue spheres whose size is the magnitude. This is the most directly actionable map the program produces: it says *this* substituent is helping and *that* one is not.

Shapley values

The *Shapley* tab decomposes a single prediction into additive contributions by the Shapley construction, over **atoms**, over **substituents of the common scaffold**, or over **field types**, with second-order interaction indices available. It serves two purposes. With the linear kernel it reproduces the atom contributions exactly, which is a useful check. With a nonlinear kernel layer, where co-fields do not exist, it is the only decomposition left — and it still works, because it needs

nothing but the ability to evaluate the model on subsets.

A pharmacophore out of the co-field

Tools → Extract pharmacophore from the co-field... turns the co-field into a discrete, typed pharmacophore: the favourable lobes of each field become feature points of the matching kind (positive and negative centres, hydrophobe, donor, acceptor), each with a tolerance radius taken from the size of its lobe, and the unfavourable steric lobes — or the pocket, when a protein is loaded — become exclusion volumes. The result is a table and a PDB-style file of feature spheres that other software can read, and it draws in the 3D view; untick *pharmacophore* in the viewer controls to clear it.

This is the bridge from a CMF model to a screening query: the model is continuous and needs an alignment, the pharmacophore is discrete and does not.

What a candidate is missing

Tools → Field difference from a reference molecule... subtracts two molecules' fields on the co-field grid and reports, per field type, the cosine between them, the distance, and how much of the reference's field the candidate **fails to reproduce** against how much it **adds**. The difference map draws as its own isosurface, and the table ranks the whole data set by field similarity to the reference. In a lead-optimisation reading: the "missing" column is what to put back, the "added" column is what may be causing trouble.

Chemical space maps

Tools → Chemical space map (GTM)... builds a two-dimensional map of the set from the CMF kernels. Fields are not vectors, so Generative Topographic Mapping cannot be applied to them directly; the program performs kernel PCA on the CMF kernel — which is functional PCA, and reproduces feature-space distances exactly — and runs the ordinary EM fit of GTM in those coordinates. The out-of-sample projection stays a single kernel row, so new molecules can be placed on an existing map.

From the responsibilities follow the **activity landscape** (responsibility-weighted mean activity per node), the **class landscape** (the fraction of actives) and the **data density**, which is a picture of the applicability domain; nodes with no support are left blank. The landscape is a model in its own right: on the thrombin training set it reproduces the activities with R^2 0.92.

Two cautions. Use cosine-normalised kernels, or the leading components follow molecular size. And a CMF kernel between molecules that share no frame is meaningless, so a map spanning several series would partly be showing alignment quality — unless you use the pose-free count kernel of *Working without an alignment*, which separates three targets' ligands cleanly with no alignment at all.

Classification and virtual screening

Setting up a classification task

Choose *Binary classification* as the task. The classes come either from a threshold on the activity (with *active below* for data where small is potent, such as an IC50 in molar units) or from a class property in the file. The learners are C-SVC, ν-SVC and the Laplace Gaussian-process classifier; the *ROC* and *Scores* tabs replace the scatter plot.

```
python cmf_cli.py build examples/sutherland/AChE.sdf --charges file \
       --task classification --threshold 6.84 --method gpc --Cs 1,10,100
```

One-class models: screening with actives alone

Often there are no reliable inactives, only a set of known actives, and the question is whether a new molecule belongs to the same class. That is one-class classification, applied to CMF by Karpov et al. (2011). Five descriptions of the active class are implemented:

Method	What it measures
`ocsvm`	the 1-SVM: a hyperplane in feature space separating the actives from the origin
`ocmean`	distance to the mean embedding of the actives, the centroid of the fields
`ocknn`	distance to the k-th nearest active
`ocgp`	the one-class Gaussian process, optionally scored by mean over standard deviation
`ockpca`	reconstruction error against the principal subspace of the actives (kernel PCA)

On the sets measured here the 1-SVM is the **most fragile** of the five: the centroid, k-NN and kernel-PCA descriptions are as good or better and have one parameter fewer to go wrong. Try more than one; they disagree in interesting places.

Decoys and the figures that matter

A screening model is only as meaningful as the background it is tested against. *File → Add decoys (background set)…* or `--decoys` loads presumed-inactive molecules used **only** for evaluation, never for fitting. `examples/make_decoy_sets.py` builds property-matched decoy sets, which is the honest kind: decoys matched to the actives in molecular weight, logP, rotatable bonds and formal charge, so that the model cannot win by noticing size.

Two traps the program handles explicitly:

- **A decoy must not be rejected for its pose.** `--align-decoys` places every decoy in the frame of the training set by maximising its field similarity, with several random restarts, and `--decoy-conformers N` generates N conformations of each decoy and keeps the one that poses best (the file's own conformation is always among the candidates).

- **AUC saturates.** With cross-target decoys almost any method reaches 0.95 and above, which tells you nothing. Read the **enrichment factors** at 0.5, 1, 5 and 10 % instead, which the program reports beside the maximum achievable value at each fraction, and **BEDROC/RIE** (Truchon & Bayly), which weight the top of the list.

A measured caution from the benchmarks in this repository: field similarity on these tasks performs about as well as ECFP4 fingerprint similarity, and cross-target decoys are trivially separable by either. If a screening result looks spectacular, check what the decoys were.

Conformations: the fourth dimension

A 3D-QSAR model represents each molecule by one conformation, and the answer depends on which one. 4D-QSAR (Hopfinger et al. 1997) replaces the choice by an ensemble and averages over it. *Model → Conformations* switches between the two; **single is the default and is untouched**, and the tests assert that by comparing its fields against those built before ensembles existed.

In CMF the averaging needs no new mathematics. A field is a sum of atom-centred Gaussians and the kernel is bilinear in the fields, so the mean field of an ensemble is itself a field — its atoms are the atoms of every conformation with their weights multiplied by that conformation's weight — and the kernel of two ensembles comes out as

$$K(A,B) = \sum_k \sum_l w_k^A w_l^B \, K\left(X_A^{(k)}, X_B^{(l)}\right)$$

the mean-map kernel of the two conformational distributions, positive definite by construction. A test checks that identity against the pairwise kernels of the conformations. The model stays linear in that mean field, so **co-fields, the decomposition rules, Shapley values and the complementarity scores all keep working**; contributions arrive per atom per conformation and fold back onto the atoms by summation.

Tools → Generate conformations... builds the ensembles with ETKDG and MMFF94s, keeping the pose a molecule already has as its first conformation, so an earlier alignment survives. A multi-conformer file can be read instead, with `--ensemble-by name` grouping the records of one compound. Weights are uniform or Boltzmann from the stored energies, with kT and a cap on the number kept.

Where the other conformations go is a choice of its own, and not a small one: what matters is not where a conformation sits relative to its own molecule but where it sits relative to the rest of the series.

`--conformer-align`	**Where each generated conformation is put**
`self`	on the molecule's own pose, over all heavy atoms — the default, so nothing changes silently
`core`	on its own pose, over the common substructure with a reference only, so the scaffold stays where the published alignment put it and only the flexible part moves
`reference`	on a reference molecule of the series, through that same common substructure, which puts every conformation of the series in one frame
`field`	as `reference`, then refined by maximising the normalised CMF kernel — the 2013 paper's criterion, applied per conformation

Multi-instance learning (`cmf_cli.py mil`) goes further: instead of averaging, the model chooses which conformation of each molecule to use, alternating between fitting and selecting. Use it when you believe one conformation is the bioactive one and the others are noise; use the ensemble when you believe the molecule is genuinely flexible.

Working without an alignment

This chapter is the one to read if your molecules came from SMILES, if your series has several scaffolds, or if you simply do not trust the superposition you have. The program offers three frame-free routes, and the scripts in `benchmarks/` measure all of them.

Why the frame is the problem

An alignment is an assumption, and a wrong one is expensive. Measured on the nine targets that occur both in the curated Sutherland collection and in the automatically aligned Mittal collection — same compounds, same activities, same fields and grids — the curated superposition gives a mean q^2 of **+0.597** and the automatic one **+0.295**. Worse, the gain over a baseline with no geometry at all (atom-type counts) is +0.25 on the curated files and **−0.06** on the automatic ones: on five of the nine targets, fields in an automatic frame predict *worse* than counts. An alignment that is not trustworthy is not merely imperfect, it is actively harmful.

The pose-free count kernel

As $\alpha \to 0$ an indicator field tends to the count of atoms of that type, so the kernel tends to the dot product of the atom-type count vectors and nothing of the geometry survives. The program implements that limit in closed form (`chemotypes.count_kernel`), which is exact rather than approached, costs one pass over the atoms, and is 10 to 34 times faster than the Gaussian route. It is the **baseline** any aligned model should be made to beat, and it is what the leave-one-chemotype-out workflow reports beside the aligned model.

One subtlety, because it is easy to get wrong: the normalisation must be applied to the joint kernel, not field by field. A field normalised on its own tends to 1 whenever both molecules contain the type at all, so the counts cancel and the baseline loses the very descriptors it is built from.

The rotation-invariant autocorrelation kernel

Geometry without a frame. For each pair of field types the program computes the smoothed field autocorrelation

$$\Phi_{fg}(d) = \sum_l \sum_m w_{lf} w_{mg} \exp\left(-\frac{(|\mathbf{r}_l - \mathbf{r}_m| - d)^2}{2\sigma^2} \right)$$

as a function of interatomic distance d, which is invariant under rotation and translation by construction, and takes the linear kernel on those channels. The $d = 0$ channel reproduces the CMF self-overlap exactly when $\sigma^2 = 2/\alpha$, so the descriptor contains the ordinary kernel's diagonal information and adds the distance profile. Select *Autocorrelation* as the kernel source in the model panel (`--kernel autocorrelation`, with `--ac-sigma` and `--ac-dmax`), and the model fits, cross-validates, saves and predicts exactly as any other — only the co-field, which needs a frame, is unavailable.

On the chemotype benchmarks the invariant kernel recovers most of the value of the geometry without a frame (0.172 against 0.189 for the aligned model and 0.072 for counts), and on automatically aligned files it frequently beats the aligned model outright — on six of the nine shared targets, including thrombin at +0.452 against +0.021.

Topomer fields

Cramer's topomer idea (topomer CoMFA, 2003), applied to fields. Cut the molecule at the bonds that attach its substituents to a common core and rebuild each substituent in a pose that is a **deterministic function of its topology**: canonical atom numbering, one embedding with a fixed seed, every rotatable torsion set to 180°, the attachment atom at the origin with its open valence along −x, and the remaining roll fixed by the principal axis of the heavy atoms. The same R-group then always lands in the same place, whatever molecule it came from — a test asserts the coordinates are identical, not merely close.

Two things make this fit CMF better than it fits CoMFA. There is no grid, so a fragment's field is just its atoms and a fragment–fragment kernel is an analytic number, which means the fragment library can be tabulated once and reused. And each attachment point keeps **its own region of space** (the substituent of position k is placed at $x = 100k$ Å, far enough that no Gaussian reaches the next site), so a molecule's kernel is exactly the sum of its per-position overlaps — which is what topomer CoMFA obtains by giving every position its own block of columns. A test checks that identity to 1e-9.

In the application it is *Tools → Replace molecules by their topomer substituents…*, and it is a **transformation of the data set** rather than a hidden switch: the molecules are replaced by their posed substituents, names, activities and the split are kept, and the data-set line says *[topomer substituents of …]*. That is the honest shape for it, because a topomer model's co-field and contributions live in the substituent frames and not around the original ligands. Reload the file to get the molecules back.

Choosing the core matters more than anything else, which is why the program now chooses it differently. The maximum common substructure of a chemotype mixture is not a scaffold — on GPB it comes out as `CCO`, fused into every ring system, and cuts nothing off at all, losing every molecule. The default is therefore the **Bemis–Murcko framework of the largest chemotype**, the scaffold of one series, with every molecule that framework comes cleanly off forming the series; on GPB that is `C1CCOCC1` and 60 of the 66 molecules. The common substructure is still available for a set you know is congeneric (*maximum common substructure of the series* in the dialog, `--core-mode mcs`), and you can always give the core yourself as SMILES or SMARTS. Check the cuttable fraction the program reports either way: a core fused to the rest of the molecule cannot be removed without opening a ring, and what is left is not a substituent.

What the three routes are worth, measured

Every column below uses the same molecules, fields, α and λ grids and folds.

Collection	Series	Aligned q^2	Topomer q^2	The cut costs	The pose costs
Sutherland, curated frame	8	**+0.445**	+0.371	+0.058	+0.015
Mittal, automatic frame	31	+0.200	**+0.256**	−0.047	−0.009
Corrosion inhibitors, neutral	17	+0.248	+0.249	+0.061	−0.061
Corrosion inhibitors, 1 M HCl	17	+0.303	+0.292	+0.056	−0.045

The last two columns separate the two things the topomer construction does at once, by way of a control that keeps the same cut substituents at the same separated sites but leaves each one

exactly where the alignment put it. The **pose rule is nearly free** — 0.015 q^2 against a careful superposition, and a *gain* of 0.045 to 0.061 where the frame is automatic, because a canonical pose is more consistent than an arbitrary embedded conformation. What costs something is the **cut**: separate sites discard the scaffold's own field and the geometry between one substituent and another, which is expensive when the arms' mutual placement is the pharmacophore (0.26 on thermolysin, 0.18 on COX-2) and free or better than free when the scaffold is constant and the decorations small (BZR gains 0.123, DHFR 0.071).

Practical guidance. With a trustworthy curated alignment, keep it: the geometry is worth about 0.25 q^2 over counts. With an automatic alignment or none, prefer the frame-free routes — topomer for a congeneric series with a clear scaffold whose R-groups come off at acyclic single bonds, and the invariant kernel for anything pooled, multi-scaffold or ring-fused. Always report the count baseline beside the model, so that the value of the geometry is visible rather than assumed.

Heterogeneous series, step by step

When a set holds several chemotypes with no one common substructure, the *Tools* → *Heterogeneous sets* submenu is a four-step workflow:

1. **Find the chemotypes** — Bemis–Murcko frameworks, optionally generic, with small groups merged so that leave-one-chemotype-out does not degenerate into leave-one-molecule-out.
2. **Align within each chemotype** — a superposition that is meaningful, because the members of one cluster do share a substructure.
3. **Place the chemotypes in one frame** — each cluster positioned by maximising field similarity to the others, which is the best that can be done when no common substructure exists.
4. **Leave one chemotype out, with the alignment-free baseline** — the honest validation, reported beside the count kernel. Random folds flatter such sets by 0.11 to 0.47 q^2 on the five chemotype benchmarks here, which is why this step exists.

Protein pockets

Getting a pocket on screen

File → Fetch protein from the PDB... downloads an entry from RCSB and caches it under `~/.cmfqsar/pdb`, with the dialog pre-filled with the identifier the data set names. *File → Load protein structure...* reads a local PDB or mmCIF file instead. The reader is deliberately independent of RDKit, because crystal structures carry alternate locations, missing elements and non-standard residues that a chemistry toolkit would refuse to sanitise, and none of that matters for drawing.

The free-energy benchmark sets in `examples/openff` and `examples/schindler` give their ligands in the frame of a reference crystal structure and name it per record, so **nothing has to be superimposed**: opening such a set brings its pocket with it from `examples/pockets/`, which holds the residues within 8 Å of any ligand, taken whole, without waters and without the co-crystallised ligand. The *protein site* controls above the 3D view then draw the residues within a chosen distance as wire frame or sticks, with hydrogens, waters and residue labels optional.

Complementarity as numbers

A pocket is a set of atoms, so the same parameterisation that builds a ligand's fields builds the protein's, and the overlap of two Gaussian fields has the same closed form on either side. The *Complementarity* tab (CLI: `cmf_cli.py complement`) turns that into three answers:

- **How complementary a molecule is to the pocket** — overlap integrals between ligand and pocket fields, exact and without a grid, each divided by the two norms so that it is a cosine in $[-1, 1]$ and does not grow with molecular size. Complementarity is not similarity: the electrostatic term carries a minus sign, because a positive ligand region should meet a negative protein region; the hydrophobic term carries a plus; the hydrogen-bond terms pair the donors of one side with the acceptors of the other. Steric overlap is reported separately as *burial*, because bulk against the protein is contact and clash at once, with the number of clashing heavy-atom pairs beside it.
- **Where on the molecule it holds** — the same quantities on a smooth van der Waals surface give a value per vertex, drawn as a green-to-red surface, with a correlation per field over the surface. Electrostatics uses potentials rather than Gaussian densities here, because the potential is what the other side feels.
- **Whether the co-field agrees with the pocket** — the model's own map against the protein, by sign agreement, cosine or correlation. This is the strongest external check a CMF model can be given: the co-field was fitted to activities and never saw the protein.

Docking

Tools → Dock into the pocket... places the conformations of a molecule in the site by rigid-body optimisation. It is a convenience for getting a pose into the right frame, not a scoring function; the program's business is what happens after the poses exist.

Command-line reference

Everything the application does is available without it, which is what makes the program scriptable for a benchmark or a cluster run. The general form is

```
python cmf_cli.py <subcommand> <dataset> [options]
```

and `python cmf_cli.py <subcommand> --help` prints the full option list; what follows is the map, not every option.

Subcommand	What it does
`build`	build, cross-validate, test and save a model
`predict`	predict with a saved model, optionally aligning by confidence first
`hetero`	the heterogeneous-series workflow: cluster, align, place, leave-one-chemotype-out
`topomer`	model a series by its substituents, posed by rule
`map`	chemical-space map and activity landscape (kernel PCA + GTM)
`explain`	Shapley decomposition of one prediction, for any kernel
`mil`	multi-instance learning over conformations
`pharmacophore`	read a pharmacophore out of a fitted model's co-fields
`difference`	what one molecule's fields have that another's do not
`complement`	field complementarity with a protein pocket
`unalign`	give every molecule a random pose, the control experiment

The options of `build`, by group

Data. `--activity PROP`, `--activity-file TABLE`, `--test-file FILE`, `--test-fraction F`, `--ensemble-by KEY`.

Fields. `--fields electrostatic,steric,hydrophobic,hba,hbd[,prop:name]`, `--cif`, `--cif-scheme sybyl|element|element+aromatic`, `--cif-min N`, `--cif-types LIST`, `--charges gasteiger|mmff|file`, `--param rdkit|tripos|tripos-legacy`, `--prop-missing error|zero`.

Kernel. `--kernel fields|autocorrelation`, `--ac-sigma S`, `--ac-dmax D`, `--kernel-layer linear|gauss|arccos1|arccos2`, `--gammas LIST`, `--normalize`, `--raw-scale`, `--optimize-h`, `--mkl [align|alignf]`, `--mkl-rho R`.

Task and learner. `--task regression|classification|oneclass`, `--method krr|gp|svr|nusvr|svc|nusvc|gpc|ocsvm|ocmean|ocknn|ocgp|ockpca`, `--threshold T`, `--active-below`, `--class-property PROP`, `--oc-k K`, `--oc-components Q`, `--oc-lam L`, `--oc-score mean|mean_over_sd`, `--no-bias`.

Hyper-parameters. `--alphas LIST`, `--lams LIST`, `--Cs LIST`, `--epsilons LIST`, `--nus LIST`, `--lam-search grid|brent`, `--alpha-search grid|nelder-mead`, `--nm-start A,L`.

Conformations. `--conformations single|ensemble`, `--conformers N`, `--conformer-window A`, `--prune-rmsd A`, `--conformer-align self|core|reference|field`, `--conformer-align-ref I`,

`--conformer-weighting uniform|boltzmann|property`, `--kt KCAL`, `--max-conformers N`.

Validation. `--folds K`, `--cv-no-shuffle`, `--nested`, `--nested-on train|all`, `--outer-folds K`, `--repeats R`, `--seed S`.

Screening. `--decoys FILE`, `--align-decoys`, `--decoy-restarts N`, `--decoy-conformers N`.

Alignment. `--align-train-variance [MAX_RMSD]`, `--align-train-rounds N`, `--align-train-folds K`.

Output. `--save model.cmf`, `--predictions out.csv`, `--json out.json`.

Worked examples

```
# indicator fields, a test split and nested cross-validation
python cmf_cli.py build examples/benzamides_synthetic.sdf --activity pActivity --cif \
       --alphas 0.3,0.5,0.8,1.2 --lams 1e-3,1e-2,0.1,1 --test-fraction 0.25 \
       --nested --save model.cmf

# predict, with the new molecules aligned by prediction confidence first
python cmf_cli.py predict model.cmf new.sdf --align-variance 2.0 \
       --aligned-out aligned.sdf --out predictions.csv

# the published protocol of the 2013 paper on a Sutherland set
python cmf_cli.py build examples/sutherland/THR.sdf --charges file --param tripos-legacy \
       --raw-scale --cv-no-shuffle --lam-search brent --nested --nested-on all

# a frame-free model: the rotation-invariant autocorrelation kernel
python cmf_cli.py build series.sdf --kernel autocorrelation --ac-sigma 0.75 --ac-dmax 12

# a topomer model with the core given explicitly
python cmf_cli.py topomer examples/benzamides_synthetic.sdf --activity pActivity \
       --core "c1ccccc1C(=O)N" --alphas 0.3,0.5,1 --lams 1e-2,0.1,1

# one-class screening against a property-matched background
python cmf_cli.py build actives.sdf --task oneclass --method ockpca \
       --decoys background.sdf --align-decoys --decoy-conformers 5

# a classification model with the Gaussian-process classifier
python cmf_cli.py build examples/sutherland/AChE.sdf --charges file --task classification \
       --threshold 6.84 --method gpc --Cs 1,10,100
```

Using it as a library

The library is the layer the application and the command-line tool are built on, and it is small enough to use directly in a notebook.

```
from cmfqsar import FieldSet, CMFModel, load_dataset
from cmfqsar.fields import AtomTable, build_fields, cif_types_in
from cmfqsar.model import grid_search

ds = load_dataset("aligned.sdf", activity_prop="pIC50")
fs = FieldSet.create(cif_types=cif_types_in(ds.mols, "sybyl", 5))
mf = [build_fields(m, fs, n) for m, n in zip(ds.mols, ds.names)]
train = AtomTable.from_molecules(mf)
model = CMFModel(fieldset=fs, method="krr")
grid_search(train, ds.activity, model, alphas=[0.3, 0.5, 1.0], lams=[1e-3, 1e-2, 0.1, 1])
model.fit(train, ds.activity, cv_folds=10, reuse_kernels=True)
print(model.stats.summary())
```

Decomposing one prediction, which works for the nonlinear kernels as well:

```
from cmfqsar.shapley import atom_shapley, field_shapley, scaffold_decomposition

res = atom_shapley(model, mf[0])                       # one value per atom
res = field_shapley(model, mf[0], interactions=True)   # one value per field type
res = scaffold_decomposition(model, mf[0], ds.mols[0], ds.mols[1:20])   # per substituent
print(res.prediction, res.baseline, res.table(), res.top_interactions(5))
```

The modules, one line each:

```
cmfqsar/fields.py       field parameterisation; FieldSet / MoleculeFields / AtomTable
cmfqsar/kernels.py      analytic field kernels, vectorised
cmfqsar/model.py        KRR / SVR / GP, cross-validation, grid search, nested CV, statistics
cmfqsar/cofields.py     co-fields and molecular fields on grids, isosurfaces, cube export
cmfqsar/dataio.py       reading SDF / MOL2 / SMILES, model persistence, exports
cmfqsar/align.py        3D embedding, MCS alignment, field-similarity refinement, unalign
cmfqsar/varalign.py     alignment by prediction confidence (GP posterior variance)
cmfqsar/invariant.py    rotation-invariant field autocorrelation kernels
cmfqsar/topomer.py      topomer fields: substituents posed by rule
cmfqsar/chemotypes.py   scaffold clusters, placement, grouped folds, the count kernel
cmfqsar/pharmacophore.py  typed features with tolerance spheres, from the co-field
cmfqsar/fielddiff.py    field differences against a reference molecule
cmfqsar/shapley.py      Shapley decomposition over atoms, substituents, field types
cmfqsar/gpc.py          Laplace Gaussian-process classification
cmfqsar/oneclass.py     centroid, k-NN, GP and kernel-PCA one-class descriptions
cmfqsar/conformers.py   conformational ensembles: generation, weighting, grouping
cmfqsar/mil.py          multi-instance learning over conformations
cmfqsar/decoys.py       posing a background set for screening evaluation
cmfqsar/gtm.py          chemical-space maps and activity landscapes
cmfqsar/protein.py      PDB / mmCIF reading, binding-site selection, frame check
cmfqsar/complementarity.py  pocket fields, complementarity scores, co-field agreement
cmfqsar/docking.py      rigid-body placement of conformations in the pocket
cmfqsar/gui/            the PySide6 application
```

The example data sets

Set	Contents
`examples/benzamides_synthetic.sdf`	109 aligned N-phenylbenzamides with a *synthetic* activity: the overlap of their fields with a hidden pseudo-receptor co-field plus noise (σ = 0.25). A model has to recover those zones; $q^2 \approx 0.87$ against a noise ceiling of 0.94. The set to learn the program on, because the right answer is known.
`examples/steroids_cbg.sdf`	the classic CoMFA benchmark: 31 steroids (Cramer, Patterson & Bunce 1988) with corticosteroid-binding-globulin affinities, in the corrected Gasteiger-group geometries, re-superimposed on the steroid skeleton of testosterone. Customary split 1–21 / 22–31, leave-one-out $q^2 \approx 0.79$.
`examples/sutherland/*.sdf`	the eight literature sets of the 2013 paper (Sutherland, O'Brien & Weaver 2004): ACE, AChE, BZR, COX-2, DHFR, GPB, THER, THR, with the aligned MMFF94s conformations as distributed, the published split, and the original Sybyl types and MNDO ESP charges as per-atom properties. Choose *Partial charges: file*.
`examples/mittal/*.sdf`	38 of the 40 benchmark sets of Mittal, McKinnon & Sorich (2009), built here from their 2D supplementary data by ETKDG and an MCS superposition, with each molecule's fit quality recorded in `align_rmsd`. The collection to use when you want to know how a method behaves on an *automatic* alignment.
`examples/openff/*.sdf`	21 congeneric series, 513 ligands, from the Open Force Field protein–ligand benchmark, each posed in its target's crystal structure; activity tag `pAffinity`.
`examples/pockets/*.pdb`	the binding sites of those series, trimmed and already in the ligands' frame, so they open together with the data.
`examples/thrombin_atom_properties.sdf`	a worked example of `prop:<name>` fields: per-atom quantum properties carried in an SDF.

`docs/benchmark.md` compares the program with Table 2 of the 2013 paper on the eight literature sets. `benchmarks/run_sutherland.py` reruns that comparison, `benchmarks/run_topomer.py` measures the alignment against the topomer rule (`--mittal` for the automatically aligned collection), `benchmarks/run_alignment.py` prices a curated alignment against an automatic one, and `benchmarks/run_corrosion.py` does the same for corrosion-inhibitor series.

When something goes wrong

The 3D view is black, or the program will not start. The viewer needs working OpenGL through PyOpenGL and pyqtgraph. On a remote desktop or in a virtual machine, software rendering is usually the fix (on Windows, a Mesa `opengl32.dll` beside the program; on Linux, `LIBGL_ALWAYS_SOFTWARE=1`). Everything except the 3D view works regardless, and the command-line tool needs no graphics at all.

q² is high but the test predictions are poor. Three usual causes, in order of frequency: the hyper-parameters were chosen on the same cross-validation you are quoting (run *External CV*); the set holds several chemotypes and random folds are flattering it (use leave-one-chemotype-out); or the alignment is doing work that will not transfer (compare against the count baseline).

q² is negative. The model predicts worse than the mean: check that the activity column is the right one and on a logarithmic scale, and that the molecules really share a frame. For R^2_{pred} on a narrow test set it may be an artefact of the variance convention, and the program reports both.

The optimum sits at the edge of the α or λ grid. Extend the grid. The *Hyper-parameter search* tab exists to make this visible.

A per-atom property is rejected. The hydrogen count almost certainly changed when the file was loaded. Write the property for all atoms including hydrogens, or pass `--prop-missing zero` deliberately.

The topomer transformation drops most of the molecules. The core is fused to the rest of them. Look at the cuttable fraction in the log and give a core joined to its R-groups by acyclic single bonds.

A decoy set gives suspiciously good enrichment. Check that the decoys are property-matched and posed in the training frame; cross-target decoys are separable by almost anything.

Co-fields are unavailable. A nonlinear kernel layer or the autocorrelation kernel source is selected, and neither is linear in the field, so the co-field does not exist. Use the *Shapley* tab instead.

It is slow. Kernel cost grows with the square of the total atom count, so the expensive sets are those with many large molecules. Cut the α grid first, since each α needs its own kernel, and remember that the count kernel is 10 to 34 times faster than the Gaussian route for a baseline.

Appendix A — Field parameterisation

Two parameterisations are available (*Increments* in the model panel, `--param`):

Field	`rdkit`, the default	`tripos`, the original CMF scripts
electrostatic	partial charge: Gasteiger, MMFF94, or read from the file	the same
steric	Tripos vdW well depth and width α/R^2_vdW, parameters by element	parameters by Sybyl atom type, Tripos 5.2 table
hydrophobic	Wildman–Crippen atomic logP contribution	in-house one-atom fragment increments by Sybyl type and attached hydrogens
H-bond acceptor / donor	approximate Abraham-type increments, donor weight on the polar hydrogen	in-house increments from Abraham's A and B constants, donor weight on the heavy atom
indicator field	1 for atoms of the given type	the same
`prop:<name>`	the value of a per-atom property carried by the file	the same

`tripos-legacy` reproduces the hydrogen-counting bug of the original R scripts, which is needed to match the published numbers exactly and should not be used for new work.

Appendix B — Files and tags the program understands

Tag or convention	Meaning
any numeric SD property	candidate activity; the loader guesses and the dialog confirms
`subset`, `set`, `split`	the train/test split, as `train`/`test` or 1/2
`atom.dprop.<name>`	a per-atom double property, written and read by RDKit
`atom.prop.sybyl_type`	per-atom Sybyl types, used automatically for indicator fields
a molecule property with one number per atom	the same per-atom property in quantum-chemistry dump form
`.cmf`	a saved model (joblib): field set, kernels, coefficients, training molecules, settings
Gaussian cube	the export format for co-fields, molecular fields, Shapley fields and difference maps
`REMARK 400` in a pocket PDB	the provenance of a trimmed binding site

Appendix C — Bibliography

References are grouped by what they are to this program: the papers that define the method it implements, those that apply it to one-class classification and virtual screening, and the work it borrows machinery or benchmark data from. Journal names follow the ACS abbreviations; a DOI is given wherever the work has one.

The method

[1] Zhokhova, N. I.; Baskin, I. I.; Bakhronov, D. K.; Palyulin, V. A.; Zefirov, N. S. Method of continuous molecular fields in the search for quantitative structure–activity relationships. *Dokl. Chem.* **2009**, *429*, 273–276. doi:10.1134/S0012500809110056

[2] Baskin, I. I.; Zhokhova, N. I. The continuous molecular fields approach to building 3D-QSAR models. *J. Comput.-Aided Mol. Des.* **2013**, *27*, 427–442. doi:10.1007/s10822-013-9656-4

[3] Baskin, I. I.; Zhokhova, N. I. Continuous molecular fields approach applied to structure–activity modeling. In *Application of Computational Techniques in Pharmacy and Medicine*; Gorb, L., Kuz'min, V., Muratov, E., Eds.; Challenges and Advances in Computational Chemistry and Physics, Vol. 17; Springer: Dordrecht, 2014; pp 433–459. doi:10.1007/978-94-017-9257-8_13

[4] Sitnikov, G. V.; Zhokhova, N. I.; Ustynyuk, Y. A.; Varnek, A.; Baskin, I. I. Continuous indicator fields: a novel universal type of molecular fields. *J. Comput.-Aided Mol. Des.* **2015**, *29*, 233–247. doi:10.1007/s10822-014-9818-z

[5] Baskin, I. I.; Zhokhova, N. I. Descriptors based on continuous indicator fields for 3D-QSAR studies. In *Proceedings of MOL2NET 2016, International Conference on Multidisciplinary Sciences*, 2nd ed.; MDPI: Basel, 2017. doi:10.3390/mol2net-02-03854

[6] Baskin, I. I.; Zhokhova, N. I. Continuous molecular fields and the concept of molecular co-fields in structure–activity studies. *Future Med. Chem.* **2019**, *11*, 2701–2713. doi:10.4155/fmc-2018-0360

One-class classification and virtual screening

[7] Karpov, P. V.; Baskin, I. I.; Zhokhova, N. I.; Zefirov, N. S. Method of continuous molecular fields in the one-class classification task. *Dokl. Chem.* **2011**, *440*, 263–265. doi:10.1134/S0012500811100016

[8] Karpov, P. V.; Baskin, I. I.; Zhokhova, N. I.; Nawrozkij, M. B.; Zefirov, A. N.; Yablokov, A. S.; Novakov, I. A.; Zefirov, N. S. One-class approach: models for virtual screening of non-nucleoside HIV-1 reverse transcriptase inhibitors based on the concept of continuous molecular fields. *Russ. Chem. Bull.* **2011**, *60*, 2418–2424. doi:10.1007/s11172-011-0372-8

Methods and data the program borrows

[9] Cramer, R. D.; Patterson, D. E.; Bunce, J. D. Comparative molecular field analysis (CoMFA). 1. Effect of shape on binding of steroids to carrier proteins. *J. Am. Chem. Soc.* **1988**, *110*, 5959–5967. doi:10.1021/ja00226a005

[10] Cramer, R. D. Topomer CoMFA: a design methodology for rapid lead optimization. *J. Med. Chem.* **2003**, *46*, 374–388. doi:10.1021/jm020194o

[11] Bemis, G. W.; Murcko, M. A. The properties of known drugs. 1. Molecular frameworks. *J. Med. Chem.* **1996**, *39*, 2887–2893. doi:10.1021/jm9602928

[12] Hopfinger, A. J.; Wang, S.; Tokarski, J. S.; Jin, B.; Albuquerque, M.; Madhav, P. J.; Duraiswami, C. Construction of 3D-QSAR models using the 4D-QSAR analysis formalism. *J. Am. Chem. Soc.* **1997**, *119*, 10509–10524. doi:10.1021/ja9718937

[13] Bishop, C. M.; Svensén, M.; Williams, C. K. I. GTM: the generative topographic mapping. *Neural Comput.* **1998**, *10*, 215–234. doi:10.1162/089976698300017953

[14] Cho, Y.; Saul, L. K. Kernel methods for deep learning. In *Advances in Neural Information Processing Systems 22 (NIPS 2009)*; Curran Associates: Red Hook, NY, 2009; pp 342–350.

[15] Rasmussen, C. E.; Williams, C. K. I. *Gaussian Processes for Machine Learning*; MIT Press: Cambridge, MA, 2006. doi:10.7551/mitpress/3206.001.0001

[16] Truchon, J.-F.; Bayly, C. I. Evaluating virtual screening methods: good and bad metrics for the "early recognition" problem. *J. Chem. Inf. Model.* **2007**, *47*, 488–508. doi:10.1021/ci600426e

[17] Sutherland, J. J.; O'Brien, L. A.; Weaver, D. F. A comparison of methods for modeling quantitative structure–activity relationships. *J. Med. Chem.* **2004**, *47*, 5541–5554. doi:10.1021/jm0497141

[18] Mittal, R. R.; McKinnon, R. A.; Sorich, M. J. Comparison data sets for benchmarking QSAR methodologies in lead optimization. *J. Chem. Inf. Model.* **2009**, *49*, 1810–1820. doi:10.1021/ci900117m